Listing 31.8 shows a simple example of a security.txt handler implemented as middleware. It always responds to the well-known path with a fixed value and is easy to add to any application by calling `app.UseMiddleware<SecurityTxtHandler>`.

Listing 31.8 A Security.txt handler implemented as middleware

```
public class SecurityTxtHandler
{
    private readonly RequestDelegate _next;
    public SecurityTxtHandler(RequestDelegate next)
    {
        _next = next;
    }

    public Task Invoke(HttpContext context)
    {
        var path = context.Request.Path;
        if(path.StartsWithSegments("/.well-known/security.txt"))  ❶
        {
            context.Response.ContentType = "text/plain";          ❷
            return context.Response.WriteAsync(                   ❷
                "Contact: mailto:security@example.com");          ❷
        }

        return _next.Invoke(context);                             ❸
    }
}
```

❶ The middleware looks for a fixed, well-known path.

❷ If the path is matched, the middleware returns a response.

❸ If the path didn’t match, the next middleware in the pipeline is called.

That covers pretty much everything you need to start building your own middleware components. By encapsulating your middleware in custom classes, you can easily test their behavior or distribute them in NuGet packages, so I strongly recommend taking this approach. Apart from anything else, it will make Program.cs file less cluttered and easier to understand.